Capacidade Efetiva de um Servidor TCP Multithread: Throughput Agregado com 1 a 10 Clientes

Disciplina: Redes de Computadores

Autores: Thiago Roberto Fernandes Rocha (RA: 149.641); Leonardo Arazo de O. Araujo (RA: 168.873); Gabriel Leal Bonavina (RA: 168.858)

Repositório: https://github.com/Arazoleo/client-server-chat

1. Introdução

Este trabalho adapta o chat TCP multithread visto em aula para medir a **capacidade efetiva máxima de recepção de dados do servidor**, com 1 a 10 clientes enviando dados continuamente ao mesmo tempo. A métrica é o **throughput agregado**, em Mbit/s: o total de bytes recebidos pelo servidor, somando todas as threads, dividido pelo tempo de medição.

O chat original continua funcionando. A diferença é um modo de teste (*benchmark*): se a primeira linha enviada por um cliente for `/bench`, o servidor não o coloca no chat nem repassa mensagens; apenas recebe e conta os bytes.

Escopo deste relatório. Todas as medições foram feitas com servidor e clientes **na mesma máquina**, comunicando-se pela interface de *loopback* (`127.0.0.1`). Em uma rede real, o resultado seria limitado também pela placa e pelo enlace (por exemplo, cerca de 1 Gbit/s em Gigabit Ethernet). Como complemento, a Seção 6 repete a bateria em uma rede **simulada** (enlaces de 1 Gbit/s e de 100 Mbit/s), também dentro da mesma máquina. Em todo o texto, a unidade é **Mbit/s** (megabits por segundo, com 1 Mbit = 10^6 bits).

2. Ambiente Experimental

A Tabela 1 resume a máquina e o ambiente. Um único notebook executou o servidor e todos os clientes, como processos distintos, dentro de uma distribuição Linux no WSL2.

Tabela 1: Ficha técnica da máquina e do ambiente.

Item	Descrição
Máquina	Notebook Lenovo, modelo 83ME
Processador	Intel Core i5-12450HX (12ª geração): 8 núcleos físicos e 12 threads; 12 CPUs lógicas visíveis ao WSL2
Memória RAM	7,7 GiB no Windows; 3,7 GiB visíveis ao WSL2
Sistema operacional (hospedeiro)	Windows 11 Pro, versão 10.0.26200
Ambiente de execução	WSL2 3.0.5.0; Ubuntu 26.04.1 LTS; kernel Linux 6.18.58.1-microsoft-standard-WSL2
Compilador	gcc 15.2.0 (Ubuntu 15.2.0-16ubuntu1), com `-O2 -Wall -Wextra -pthread`
Rede	*Loopback* (`lo`, `127.0.0.1`), MTU 65 536. Sem placa de rede física no caminho dos dados
Parâmetros TCP do Linux	controle de congestionamento *cubic*; *timestamps* ativos; `rmem_max` e `wmem_max` de 4 MiB

3. Metodologia

3.1. Modificações no código

Servidor (`chat_server.c`). Cada conexão é atendida por uma thread. A thread lê a primeira linha. Se for `/bench`, entra num laço que chama `recv()` com um buffer de 256 KiB e soma, numa variável própria da thread e sem *mutex* dentro do laço, o número de bytes que o `recv()` realmente devolveu. A thread anota, com `CLOCK_MONOTONIC`, o instante do primeiro e do último `recv()` bem-sucedido. Quando o cliente fecha o seu lado (`recv()` devolve 0), a thread soma seus bytes ao total global, sob

mutex, e atualiza o instante mais cedo e o mais tarde da rodada. A **última thread da rodada** calcula e imprime a taxa agregada:

```
BENCH clients=3 bytes=3000000000 seconds=6.731 mbps=3565.60
```

O servidor também imprime uma linha `BENCH_CLIENT` por conexão, com os bytes que ela recebeu. O `listen()` passou de 10 para 128, para que 10 clientes conectando juntos não dependam do tamanho da fila.

Cliente (`chat_client.c`). O modo `-bench <segundos> <bloco> [-start-at <epoch>]` conecta, envia `/bench\n` e, sem usar o teclado, envia blocos de tamanho fixo em laço até o tempo acabar. Em seguida chama `shutdown(fd, SHUT_WR)`, espera o servidor fechar a conexão e imprime `BENCH_SENT bytes=... seconds=....`

Largada sincronizada. Com `-start-at`, o cliente conecta, envia `/bench` e dorme até o horário combinado. Se o horário já passou, ou se o cliente acordar mais de 0,1 segundo depois dele, o cliente sai com erro, porque aquela rodada não seria comparável.

Automação (`run_bench.sh`). Para cada quantidade de clientes de 1 a 10 e cada repetição, o script escolhe um horário 2 a 3 segundos no futuro, dispara todos os clientes com o mesmo `-start-at`, espera todos terminarem e faz uma pausa de 2 segundos antes da rodada seguinte. Ele registra o que cada cliente enviou em `clients.log`.

3.2. Parâmetros da bateria

- **Clientes simultâneos:** de 1 a 10, todos na mesma máquina do servidor.
- **Repetições:** 5 por quantidade de clientes, totalizando 50 rodadas.
- **Duração do envio:** 10 segundos por rodada.
- **Tamanho do bloco:** 65 536 bytes (64 KiB) em todas as rodadas.
- **Conteúdo enviado:** bytes constantes; o conteúdo não importa para o servidor.
- **Comando:** `./run_bench.sh 127.0.0.1 5000 10 65536 5 10`, com o servidor iniciado antes em outro processo.

3.3. Como a taxa agregada é calculada

Seja b_i o total de bytes que a thread i recebeu de seu cliente, t_i^{ini} e t_i^{fim} os instantes do primeiro e do último `recv()` dessa thread, e N o número de clientes da rodada:

$$R \;=\; \frac{8 \sum_{i=1}^{N} b_i}{\left(\max_i t_i^{\text{fim}} - \min_i t_i^{\text{ini}}\right) \cdot 10^6} \quad \text{[Mbit/s]}. \tag{1}$$

O numerador é o total de bytes **efetivamente recebidos** pelo servidor em todas as threads, e não o que os clientes tentaram enviar. O denominador é a janela do primeiro byte de *qualquer* cliente ao último byte de *qualquer* cliente, de modo que o tempo de conexão e a espera antes da largada não entram na conta. Só os dados da aplicação são contados.

3.4. Garantias de comparabilidade

- Todos os clientes de uma rodada começam no mesmo instante (largada sincronizada) e rodam pelo mesmo tempo, com o mesmo tamanho de bloco em todas as rodadas.
- O servidor contabiliza cada rodada separadamente e só finaliza a conta quando o último cliente termina.
- Antes de gerar qualquer tabela, o script confere os dois logs rodada a rodada e **recusa** os dados se: o servidor viu menos clientes do que o script disparou; algum cliente não terminou; o total de

bytes recebido pelo servidor difere do total que os clientes enviaram; há rodadas de outra bateria no mesmo `server.log`; ou as quantidades de clientes têm números de repetições diferentes. Os logs desta bateria foram aceitos por essa verificação.

- O servidor foi iniciado uma única vez, com um `server.log` novo, e todas as 50 rodadas rodaram em sequência, sem intervenção. O desvio padrão é o amostral (5 repetições por ponto).
- O código foi compilado com as mesmas opções em todo o experimento, e a suíte de testes do repositório (`tests/test_bench.sh`) passou sem falhas antes da bateria.
- Por ser um notebook, o computador foi mantido na tomada durante a bateria. Na bateria principal havia alguns programas leves abertos, mas nenhum pesado.

3.5. Perfil de uso da CPU

Para discutir gargalos, foram feitas rodadas adicionais e isoladas (porta 5001, mesmo cliente e servidor, 10 s, bloco de 64 KiB) com 1, 3, 4, 5 e 10 clientes, enquanto o `vmstat 1` registrava o uso da CPU de toda a máquina virtual. A Tabela 4 traz a média das amostras em que a CPU estava ocupada (usuário + sistema acima de 5%). Essas rodadas **não** entram na tabela de resultados.

4. Resultados

A Tabela 2 e a Figura 1 mostram o throughput agregado de 1 a 10 clientes (5 repetições de 10 s cada).

Tabela 2: Throughput agregado recebido pelo servidor, em loopback. "Ganho" é a média dividida pela média de 1 cliente.

Clientes	Média (Mbit/s)	Desvio (Mbit/s)	Mín. (Mbit/s)	Máx. (Mbit/s)	GB/s (média)	Mbit/s por cliente	Ganho (×)
1	51 401,08	540,60	50 618,62	51 977,53	6,43	51 401	1,00
2	107 568,62	965,23	106 125,33	108 548,21	13,45	53 784	2,09
3	159 565,39	2 998,53	156 606,44	164 460,76	19,95	53 188	3,10
4	114 556,29	31 514,10	76 873,81	154 026,94	14,32	28 639	2,23
5	70 670,72	2 174,89	68 122,84	73 617,80	8,83	14 134	1,37
6	69 544,36	820,79	68 561,34	70 704,75	8,69	11 591	1,35
7	70 971,90	482,19	70 487,84	71 654,80	8,87	10 139	1,38
8	71 969,71	634,60	70 918,02	72 519,06	9,00	8 996	1,40
9	70 009,60	904,01	68 583,03	70 827,64	8,75	7 779	1,36
10	68 854,00	680,69	67 982,25	69 658,26	8,61	6 885	1,34

Observações.

1. **De 1 a 3 clientes, o total cresce de forma quase linear.** A média passa de 51,4 para 107,6 e para 159,6 Gbit/s; cada cliente adicional soma cerca de 53 Gbit/s. O **máximo do experimento** foi com 3 clientes: 159,6 Gbit/s em média e 164,5 Gbit/s na melhor repetição.
2. **Com 4 clientes o resultado é instável.** As cinco repetições deram 117,3; 134,4; 76,9; 90,2 e 154,0 Gbit/s, um desvio padrão de 31,5 Gbit/s (cerca de 28% da média). Por isso a média de 114,6 Gbit/s não representa bem esse ponto. Nas rodadas isoladas do perfil de CPU, 4 clientes deram valores estáveis de 87 a 88 Gbit/s, o que reforça que esse cenário é sensível a condições do momento.
3. **De 5 a 10 clientes o total se estabiliza num patamar baixo.** O throughput fica entre 68,0 e 73,6 Gbit/s, com médias de 68,9 a 72,0 Gbit/s, só 1,34× a 1,40× o de um único cliente e menos da metade do pico de 3 clientes. Com **10 clientes**, a média é 68,9 Gbit/s (≈8,6 GB/s), com desvio de 0,7 Gbit/s.
4. **A banda por cliente cai.** Com 10 clientes, cada um recebe em média cerca de 6,9 Gbit/s.

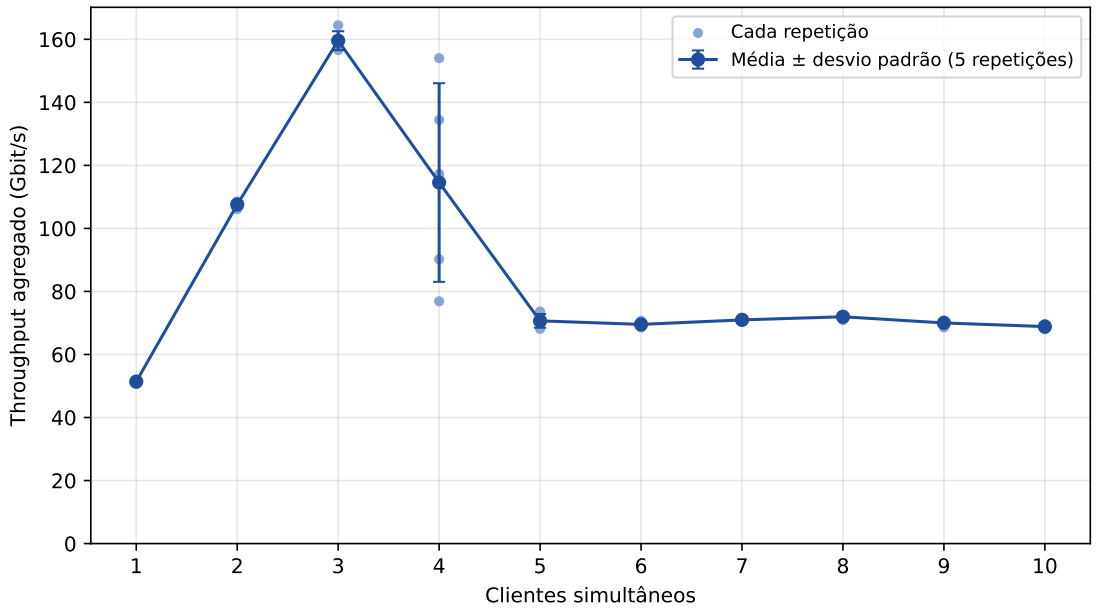


Figura 1: Throughput agregado em função do número de clientes. Os pontos claros são as 5 repetições de cada quantidade; a linha é a média, com o desvio padrão como barra de erro.

5. **Divisão balanceada entre clientes.** A razão entre o cliente que mais e o que menos recebeu numa rodada foi de no máximo 1,38 (Figura 2).

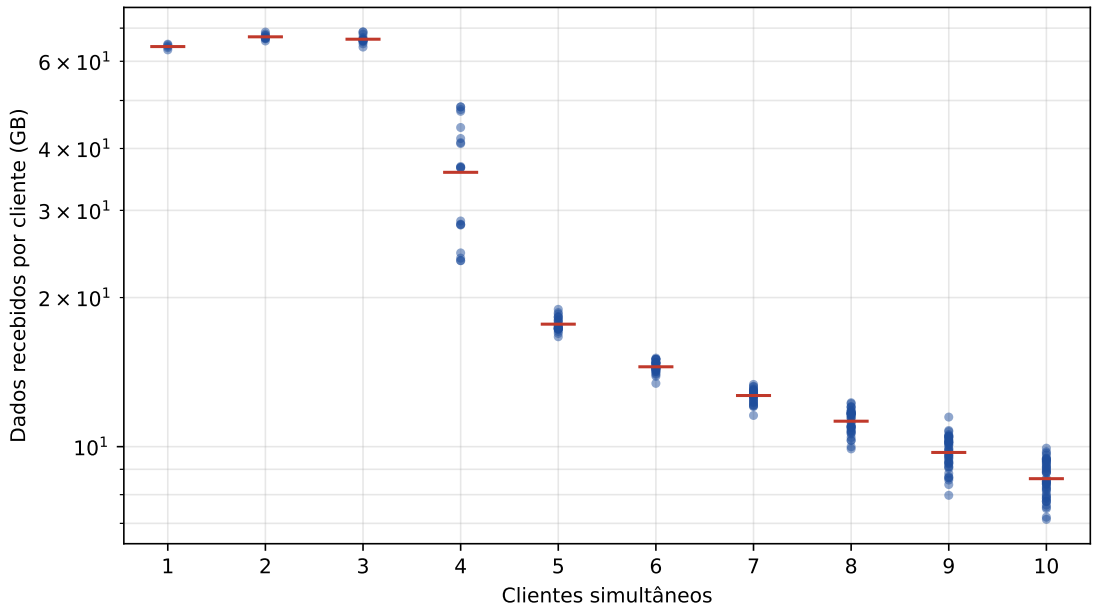


Figura 2: Dados recebidos por cliente em cada rodada (pontos azuis) e média por cliente (traço vermelho), em escala logarítmica.

4.1. Repetição da bateria

Para medir a repetibilidade, a bateria completa (50 rodadas, mesmos parâmetros e mesmo procedimento) foi executada uma segunda vez, no mesmo dia. A Tabela 3 e a Figura 3 comparam as duas baterias.

- **A forma da curva se repetiu.** Com 1 e 2 clientes e de 6 a 10 clientes, as médias das duas baterias diferem em no máximo 1,4%. Com 10 clientes, que é o cenário principal, foram 68,9 e

Tabela 3: Bateria principal e repetição: média e desvio padrão do throughput agregado (Gbit/s) e diferença relativa da média.

Clientes	Principal	Desvio	Repetição	Desvio	Diferença (%)
1	51,4	0,5	50,7	0,4	−1,4
2	107,6	1,0	107,3	0,7	−0,2
3	159,6	3,0	156,2	7,1	−2,1
4	114,6	31,5	99,1	19,1	−13,5
5	70,7	2,2	76,4	6,1	+8,1
6	69,5	0,8	68,8	0,4	−1,0
7	71,0	0,5	70,1	0,3	−1,2
8	72,0	0,6	71,3	0,6	−0,9
9	70,0	0,9	70,8	0,6	+1,1
10	68,9	0,7	69,5	0,2	+0,9

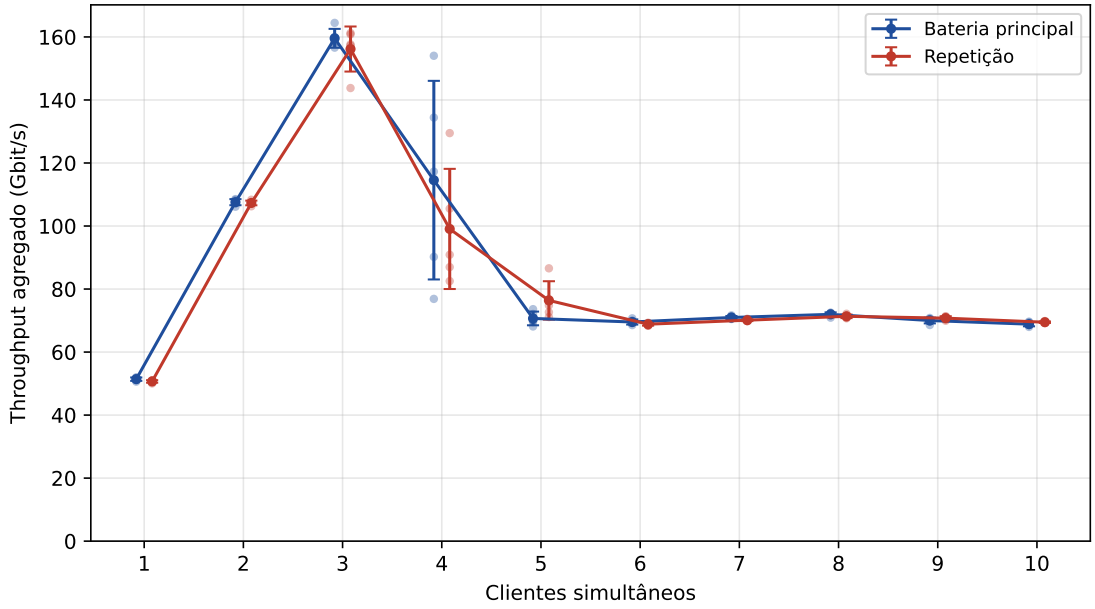


Figura 3: Bateria principal (azul) e repetição (vermelho). Pontos claros: cada repetição; linhas: média com desvio padrão.

69,5 Gbit/s (0,9% de diferença). O pico em 3 clientes também se repetiu: 159,6 e 156,2 Gbit/s (−2,1%).

- **Os pontos de 4 e 5 clientes são instáveis.** Com 4 clientes, as repetições variaram de 76,9 a 154,0 Gbit/s na bateria principal e de 82,5 a 129,5 Gbit/s na repetição, com médias de 114,6 e 99,1 Gbit/s (diferença de 13,5%): o valor depende da rodada, e só se pode afirmar que fica entre o patamar e o pico. Com 5 clientes, a repetição teve valores de 71,1 a 86,6 Gbit/s (média 8,1% acima da bateria principal), então esse ponto também está numa região de transição; o patamar fica bem estável a partir de 6 clientes nas duas baterias.

5. Discussão: Limites e Gargalos

5.1. Kernel limitante

Na Tabela 4, o tempo de CPU em modo usuário (*us*) é desprezível, cerca de 1%, em todos os cenários. Quase toda a CPU é gasta em modo kernel (*sy*), e esse valor cresce com o número de clientes: de cerca de 13% com 1 cliente para 43%, 56%, 70% e 81% com 3, 4, 5 e 10. O servidor só chama `recv()` em laço e soma números; o trabalho caro está na pilha TCP do sistema operacional, que copia os dados do cliente para o kernel e do kernel para o servidor. Com 10 clientes a CPU fica com apenas 17% de ociosidade, ou seja, a máquina está perto da saturação, e é isso que impede o total de crescer.

Tabela 4: Uso médio da CPU (`vmstat`, toda a máquina virtual) nas amostras em que ela estava ocupada. *us*: código de usuário; *sy*: kernel; *id*: ociosa.

Clientes	Mbit/s (rodada)	Amostras	us (%)	sy (%)	id (%)
1	50 672	11	0,3	13,5	86,1
3	129 331	11	1,0	42,6	56,4
4	88 270	11	0,9	56,4	42,9
4	86 905	11	1,0	56,5	42,6
4	87 725	11	1,0	56,5	42,6
5	68 892	11	1,0	69,6	29,6
10	66 600	12	1,1	81,2	17,3

5.2. Queda no valor total após três clientes

Cada cliente adicional ocupa mais duas threads (a que envia e a que recebe, mais o trabalho do kernel entre as duas), e o notebook tem 12 CPUs lógicas que compartilham núcleos, caches e memória. O pico com 3 clientes e a queda para um patamar de cerca de 70 Gbit/s a partir de 5 sugerem que, passado certo número de threads ativas, o ganho de paralelismo é superado pelo custo de disputa por recursos.

5.3. Limites do experimento

- **Não é uma medição de rede.** Por rodarmos na mesma máquina, o *loopback* elimina a placa e o enlace.
- **Servidor e clientes disputam a mesma CPU.** Parte do que seria capacidade do servidor é gasta pelos próprios clientes de teste, que também usam a CPU do sistema para enviar.
- **O WSL2 é uma máquina virtual** sobre o Windows. O desempenho depende de como o hospedeiro distribui CPU e memória, e o ambiente tem outros processos do Windows rodando, fora do nosso controle. O WSL2 enxergava 3,7 GiB de RAM.
- **Repetibilidade.** A repetição da bateria (Seção 4.1) mostrou diferenças de no máximo 2% nos pontos estáveis (1, 2, 3 e de 6 a 10 clientes), mas de 8% e 13% com 5 e 4 clientes. Numa rodada isolada do perfil de CPU, 3 clientes deram 129,3 Gbit/s, abaixo das duas baterias (159,6 e 156,2 Gbit/s), o que mostra que o valor absoluto pode variar entre execuções; a forma geral da curva se manteve.
- **Resultados específicos desta máquina.** Em outro computador, as quantidades de clientes em que o pico e o patamar aparecem tendem a ser diferentes.

6. Experimento Complementar: Rede Simulada

Para observar o servidor quando o limite é o enlace e não a CPU, a bateria foi repetida em uma **rede simulada**, com enlaces de 1 Gbit/s e de 100 Mbit/s.

6.1. Montagem

- **Dois "computadores virtuais".** Dois *namespaces* de rede do Linux: `ns_srv` (10.0.0.1), onde roda o servidor, e `ns_cli` (10.0.0.2), onde rodam os clientes, ligados por um par de interfaces virtuais (`veth`). O tráfego atravessa a pilha TCP/IP completa de cada lado.
- **Limite de taxa.** O `tc` com a disciplina `tbf` limita a taxa nos dois sentidos: 1 Gbit/s (*burst* de 1 MB) e 100 Mbit/s (*burst* de 128 KB). A fila de cada lado comporta no máximo 20 ms de transmissão na taxa configurada e descarta o excesso. MTU de 1500 bytes. A segmentação por *offload* (TSO, GSO e GRO) foi desligada, para que o `tc` enxergue pacotes de 1500 bytes, como numa placa real.

- **Mesmo experimento.** Mesmo código, mesmo script e mesmos parâmetros da bateria principal (1 a 10 clientes, 5 repetições, 10 s, blocos de 64 KiB). A configuração está em `rede_simulada.sh`. O ambiente (Tabela 1) é o mesmo.

6.2. Resultados

Tabela 5: Rede simulada de **1 Gbit/s**: throughput agregado e divisão entre clientes (5 repetições). "Razão máx./mín." é a maior razão, entre as repetições, entre o cliente que mais e o que menos enviou.

Clientes	Média (Mbit/s)	Desvio (Mbit/s)	Mín. (Mbit/s)	Máx. (Mbit/s)	Mbit/s por cliente	Razão máx./mín.
1	957,21	0,05	957,14	957,27	957,2	1,00
2	957,20	0,03	957,15	957,21	478,6	3,28
3	957,16	0,03	957,11	957,19	319,1	2,57
4	957,19	0,01	957,17	957,21	239,3	2,66
5	957,18	0,04	957,11	957,21	191,4	1,67
6	957,19	0,06	957,13	957,27	159,5	1,12
7	957,17	0,05	957,10	957,22	136,7	1,63
8	957,15	0,06	957,07	957,24	119,6	1,12
9	957,01	0,34	956,40	957,19	106,3	1,10
10	957,17	0,05	957,10	957,23	95,7	1,10

Tabela 6: Rede simulada de **100 Mbit/s**: mesmos campos da Tabela 5.

Clientes	Média (Mbit/s)	Desvio (Mbit/s)	Mín. (Mbit/s)	Máx. (Mbit/s)	Mbit/s por cliente	Razão máx./mín.
1	95,74	0,00	95,74	95,74	95,7	1,00
2	95,73	0,01	95,73	95,74	47,9	1,40
3	95,73	0,01	95,72	95,74	31,9	2,20
4	95,73	0,01	95,71	95,74	23,9	3,32
5	95,73	0,01	95,72	95,74	19,1	5,15
6	95,73	0,00	95,73	95,73	16,0	5,11
7	95,73	0,01	95,71	95,74	13,7	5,73
8	95,73	0,01	95,71	95,74	12,0	2,64
9	95,73	0,01	95,72	95,74	10,6	8,66
10	95,74	0,00	95,73	95,74	9,6	6,32

1. **O total é constante de 1 a 10 clientes.** A média fica entre 957,0 e 957,2 Mbit/s no enlace de 1 Gbit/s (desvio de no máximo 0,34) e entre 95,73 e 95,74 Mbit/s no de 100 Mbit/s (desvio de no máximo 0,01). Um único cliente já satura o enlace, ao contrário do *loopback*, onde o total crescia até 3 clientes.

2. **Enlace é o gargalo.** Com o enlace limitado a uma fração do que o *loopback* alcançou (51 Gbit/s com 1 cliente), é de se esperar que a CPU não seja o limite.

3. **A divisão entre clientes é menos equilibrada que em *loopback*.** Em *loopback*, o cliente que mais enviou nunca passou de 1,38 vez o que menos enviou. Na rede simulada essa razão chega a 3,28 (1 Gbit/s) e a 8,66 (100 Mbit/s). O enlace fixa o total, mas não a parte de cada cliente: os fluxos TCP disputam a mesma fila do enlace, nada no servidor impõe uma divisão igual e, de uma rodada para outra, a fatia de cada cliente muda.

4. **Perdas.** Segundo o `tc -s` ao fim da bateria, a fila do lado dos clientes descartou 5 164 pacotes dos 4,25 milhões enviados no enlace de 100 Mbit/s (0,12%), e nenhum no de 1 Gbit/s, onde o `tbf` apenas atrasou os pacotes (cerca de 2,8 milhões de ocorrências de *overlimit*).

Interpretação. Com um enlace limitado, o servidor recebe sempre cerca de 100% do que o enlace entrega e o número de clientes só muda como essa banda é dividida, o que confirma o comportamento esperado em uma rede real e contrasta com o *loopback*, em que o limite era a CPU.

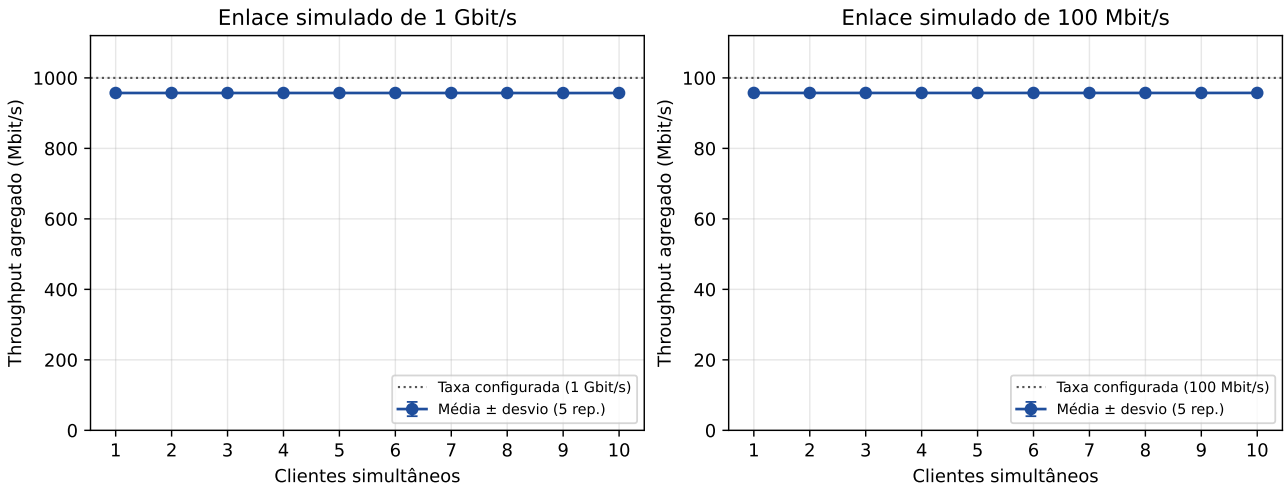


Figura 4: Throughput agregado nos enlaces simulados de 1 Gbit/s e de 100 Mbit/s. Pontos claros: cada repetição; linha: média com desvio padrão.

7. Conclusão

Com servidor e clientes na mesma máquina, em *loopback*, o throughput agregado recebido pelo servidor cresceu de 51,4 Gbit/s com 1 cliente até um pico de 159,6 Gbit/s com 3 clientes, ficou instável com 4 e se estabilizou em cerca de 68 a 73 Gbit/s de 5 a 10 clientes (a repetição da bateria confirmou a forma da curva, com 4 e 5 clientes como os pontos de maior incerteza). Com 10 clientes, a média foi de 68,9 Gbit/s (≈8,6 GB/s). O gargalo observado é a CPU do sistema, gasta quase toda em modo kernel na pilha TCP, e não o código do servidor, que passa menos de 2% do tempo em modo usuário. Mais clientes não aumentam o total depois do pico, mas a banda continua dividida de forma justa entre eles. Em uma rede simulada, o total ficou fixo na capacidade do enlace (cerca de 957 Mbit/s em 1 Gbit/s e 95,7 Mbit/s em 100 Mbit/s) de 1 a 10 clientes: o gargalo passa para o enlace e os clientes apenas dividem a banda, de forma menos bem distribuída do que em *loopback*.

8. Código-fonte, Compilação e Execução

Repositório com o servidor e o cliente modificados, o script de automação e os testes: https://github.com/Arazoleo/client-server-chat. Os logs brutos desta bateria, o script de figuras e o perfil de CPU estão em `relatorio/dados/` no mesmo repositório.

```
# compilar
gcc -O2 -Wall -Wextra chat_server.c -o chat_server -pthread
gcc -O2 -Wall -Wextra chat_client.c -o chat_client -pthread

# servidor (um terminal)
./chat_server 5000 > server.log

# clientes (outro terminal): IP, porta, segundos, bloco, repeticoes, max. clientes
./run_bench.sh 127.0.0.1 5000 10 65536 5 10

# gerar as tabelas a partir dos logs
./run_bench.sh --csv server.log clients.log > results.csv
./run_bench.sh --summary server.log clients.log > summary.csv

# testes automatizados (em loopback)
tests/test_bench.sh

# rede simulada (como root; troque 1gbit 1mb por 100mbit 128kb para o outro cenario)
sudo ./rede_simulada.sh setup 1gbit 1mb
sudo ./rede_simulada.sh exec srv ./chat_server 5000 > server.log &
sudo ./rede_simulada.sh exec cli ./run_bench.sh 10.0.0.1 5000 10 65536 5 10
```

```
sudo ./rede_simulada.sh teardown

# figuras e tabelas deste relatorio (logs em relatorio/dados/)
python relatorio/make_figures.py
```

O chat normal continua funcionando com `./chat_client <IP> <porta>`. Para repetir em duas máquinas, basta iniciar o servidor em uma delas e rodar o `run_bench.sh` na outra com o IP do servidor.

9. Contribuição de cada integrante

- **Leonardo Arazo de O. Araujo:** servidor (`chat_server.c`): modo `/bench`, contagem de bytes, cálculo da taxa agregada, `listen()` com 128 e análise dos gargalos.
- **Gabriel Leal Bonavina:** cliente (`chat_client.c`): modo `-bench`, largada sincronizada, `shutdown()`, automação (`run_bench.sh`) e análise dos gargalos.
- **Thiago Roberto Fernandes Rocha:** relatório, ficha técnica, execução da bateria e do perfil de CPU, revisão do código e análise dos gargalos.